

C++ QUICK REFERENCE / C++ CHEATSHEET

C++ Data Types

Data Type	Description
bool	boolean (true or false)
char	character ('a', 'b', etc.)
char[]	character array (C-style string if null terminated)
string	C++ string (from the STL)
int	Integer (1, 2, -1, 1000, etc.)
long int	long integer
float	single precision floating point
double	double precision floating point

These are the most commonly used types: this is not a complete list.

Operators

The most commonly used operators in order of precedence:

- ➊ ++ (post-increment), -- (post-decrement)
- ➋ !(not), ++ (pre-increment), -- (pre-decrement)
- ➌ *, %(modulus)
- ➍ +-
- ➎ <, <=, >, >=
- ➏ == (equal-to), != (not-equal-to)
- ➐ && (and)
- ➑ || (or)
- ➒ = (assignment), *, /, % =, + =, - =

Looping

while Loop

```
while (expression)
    statement;
```

Example:

```
while (x < 100)
    cout << x++ << endl;
```

```
while (expression)
{
    statement;
    statement;
}
```

Example:

```
while (x < 100)
{
    cout << x << endl;
    x++;
}
```

do-while Loop

```
do
    statement;
while (expression);
```

Example:

```
do
    cout << x++ << endl;
while (x < 100);
```

```
do
{
    statement;
    statement;
}
while (expression);
```

Example:

```
do
{
    cout << x << endl;
    x++;
}
while (x < 100);
```

for Loop

```
for (initialization; test; update)
    statement;
```

```
for (initialization; test; update)
{
    statement;
    statement;
}
```

Example:

```
for (count = 0; count < 10; count++)
{
    cout << "count equals: " << count << endl;
}
```

Decision Statements

if

```
if (expression)
    statement;
```

Example:

```
if (x < y)
    cout << x;
```

if/else

```
if (expression)
    statement;
else
    statement;
```

Example:

```
if (x < y)
    cout << x;
else
    cout << y;
```

switch/case

```
switch(int expression)
{
    case int-constant:
        statement(s);
        break;
    case int-constant:
        statement(s);
        break;
    default:
        statement;
}
```

Example:

```
switch(choice)
{
    case 0:
        cout << "Zero";
        break;
    case 1:
        cout << "One";
        break;
    default:
        cout << "What?";
}
```

Pointers

A pointer variable (or just pointer) is a variable that stores a memory address. Pointers allow the indirect manipulation of data stored in memory. Pointers are declared using *. To set a pointer's value to the address of another variable, use the & operator.

Example:

```
char c = 'a';
char* cPtr;
cPtr = &c;
```

Use the indirection operator (*) to access or change the value that the pointer references. Example:

```
// continued from example above
*cPtr = 'b';
cout << *cPtr << endl; // prints the char b
// prints the char b
```

Array names can be used as constant pointers, and pointers can be used as array names. Example:

```
int numbers[] = {10, 20, 30, 40, 50};
int* numPtr = numbers;
cout << numbers[0] << endl; // prints 10
cout << *numPtr << endl; // prints 10
cout << numbers[1] << endl; // prints 20
cout << *(numPtr + 1) << endl; // prints 20
cout << numPtr[2] << endl; // prints 30
```

Dynamic Memory

Allocate Memory

Syntax	Examples
ptr = new type;	int* iPtr; iPtr = new int;
ptr = new type [size];	int* intArray; intArray = new int[5];

Deallocate Memory

Syntax	Examples
delete ptr;	delete iPtr;
delete [] ptr;	delete [] intArray;

Once a pointer is used to allocate the memory for an array, array notation can be used to access the array locations. Example:

```
int* intArray;
intArray = new int[5];
intArray[0] = 23;
```

Classes

Declaration

```
class classname
{
public:
    classname (params);
    ~classname();
    type member1;
    type member2;
protected:
    type member3;
private:
    type member4;
};
```

Example:

```
class Square
{
public:
    Square();
    Square (float w);
    void setWidth(float w);
    float getArea();
private:
    float width;
};
```

- **public** members are accessible from anywhere the class is visible.
- **private** members are only accessible from the same class or a friend (function or class).
- **protected** members are accessible from the same class, derived classes, or a friend (function or class).
- constructors may be overloaded just like any other function. You can define two or more constructors as long as each constructor has a different parameter list.

Definition of Member Functions

```
return_type classname::functionName (params)
{
    statements;
}
```

Examples:

```
Square::Square()
{
    width = 0;
}
void Square::setWidth(float w)
{
    if (w >= 0)
        width = w;
    else
        exit(-1);
}
float Square::getArea()
{
    return width * width;
}
```

Definition of Instances

```
classname varName;
classname* ptrName;
```

Example:

```
Square s1();
Square s2(3.5);
Square* sPtr;
sPtr = new Square (1.8);
```

Accessing Members

```
varName.member = val;
varName.member();
ptrName->member = val;
ptrName->member();
```

Example:

```
s1.setWidth(1.5);
cout << s1.getArea();
cout << sPtr->getArea();
```

Inheritance

Inheritance allows a new class to be based on an existing class. The new class inherits all the member variables and functions (except the constructors and destructor) of the class it is based on. Example:

```
class Student
{
public:
    Student (string n, string id);
    void print();
protected:
    string name;
    string netID;
};
```